

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	PATTEELA YASHWANTH KUMAR	Reg.No.:	192471055
Department:	CSE-BIOSCIENCE	Date:	01-08-2026

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I Patteela Yashwanth Kumar/192471055 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Patteela Yashwanth kumar

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation

Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided
---------------------------	-----	--	---	--	---

Question 1

A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

Answer

1. Problem Identification

- A program performing signed integer addition in **8-bit 2's complement** gives an incorrect result while adding **+120 and +50**.
- Decimal addition:
120 + 50 = 170
- However, an 8-bit signed integer can represent only **-128 to +127**. Therefore, **170 cannot be represented**, causing overflow.

2. Binary Analysis

```

+120 = 01111000
+50 = 00110010
      01111000
+   00110010
-----
      10101010

```

- The result **10101010** represents **-86** in 2's complement, which is incorrect.

3. Error Identification

The error is **signed overflow**.

Overflow occurs because:

- Both operands are positive.
- The result becomes negative.
- The carry into the sign bit is different from the carry out of the sign bit.

4. Debugging Approach

- Check operand signs before addition.
- Compare carry into and carry out of the MSB.
- If they differ, set an overflow flag.
- Prevent the incorrect result from being used.

5. Correct Solution

Implement overflow detection:

If (Carry into MSB $\neq$ Carry out of MSB)

Overflow = 1

Else

Overflow = 0

When overflow occurs:

- Display an overflow message.

- Use a larger data type (16-bit or 32-bit integer).
- Stop further computation if required.

6. Optimization

- Use a processor's built-in overflow flag.
- Perform addition using 16-bit registers when large values are expected.
- Reduce arithmetic errors by checking overflow immediately after addition.

7. Testing and Validation

Test Case Expected Result

20 + 30	50
100 + 20	120
120 + 50	Overflow detected
-100 + -40	Overflow detected

8. Conclusion

The issue is caused by **signed overflow**. Detecting overflow using the carry or sign-bit method ensures accurate arithmetic and prevents incorrect results.

Question2

A processor using a ripple carry adder shows significant delay in executing arithmetic operationa real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

1. Problem Identification

- A processor using a **Ripple Carry Adder (RCA)** experiences significant delay while performing arithmetic operations in a real-time system.
- The delay occurs because the carry generated at one bit must propagate through every subsequent bit before the final sum is produced.

2. Debugging Approach and Analysis

In a Ripple Carry Adder:

- Each full adder waits for the carry from the previous stage.
- The carry propagates sequentially from the Least Significant Bit (LSB) to the Most Significant Bit (MSB).
- As the number of bits increases, the propagation delay also increases.
- This makes the processor slower and unsuitable for high-speed or real-time applications.

Example (4-bit addition):

A = 1011

B = 0101

FA0 → Carry0

↓

FA1 → Carry1

↓

FA2 → Carry2



FA3 → Final Carry

- The final result is available only after the carry passes through all four full adders.

3. Error Identification

The performance bottleneck is **carry propagation delay**.

Root Cause:

- Carry is generated one stage at a time.
- Every stage depends on the previous carry.
- Total delay increases linearly with the number of bits.

4. Correct Debugged Solution

Replace the **Ripple Carry Adder** with a **Carry Look-Ahead Adder (CLA)**.

The CLA computes all carry signals simultaneously using **Generate (G)** and **Propagate (P)** signals.

Generate:

- $G = A \times B$

Propagate:

- $P = A \oplus B$

Using these signals, carries are calculated in parallel instead of waiting for the previous carry.

5. Optimization Technique

Advantages of Carry Look-Ahead Adder:

- Computes carry bits simultaneously.
- Greatly reduces propagation delay.
- Improves processor speed.
- Suitable for high-speed arithmetic operations.
- Enhances overall system performance in real-time applications.

6. Testing and Validation

Test Case Expected Result

5 + 3	Correct output with low delay
12 + 9	Faster than Ripple Carry Adder
15 + 15	Correct result with reduced carry propagation time

7. Conclusion

The delay is caused by **sequential carry propagation** in the Ripple Carry Adder. Replacing it with a **Carry Look-Ahead Adder** allows carry signals to be generated in parallel, significantly reducing delay and improving processor performance for real-time systems

Question 3

A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

1. Problem Identification

A multiplication unit using **Booth's Algorithm** gives incorrect results when multiplying **negative signed numbers**.

Example:

- Multiplicand (M) = **-5**
- Multiplier (Q) = **3**

The expected result is:

$$-5 \times 3 = -15$$

However, due to errors in the algorithm, the computed result is incorrect.

2. Debugging Approach and Analysis

Booth's Algorithm performs multiplication by examining the last bit of the multiplier (Q_0) and an extra bit (Q_{-1}).

Step 1: Check Bit Pair

$Q_0 Q_{-1}$ Operation

00	No operation
01	Add Multiplicand (M)
10	Subtract Multiplicand (M)
11	No operation

Step 2: Perform Arithmetic Right Shift

- Shift the Accumulator (A), Multiplier (Q), and Q_{-1} together.
- Preserve the sign bit during the shift.

Step 3: Repeat

- Repeat the above steps for **n** bits.

3. Error Identification

The incorrect result may occur due to:

- Incorrect bit-pair (Q_0, Q_{-1}) checking.
- Logical right shift used instead of arithmetic right shift.
- Failure to preserve the sign bit during shifting.
- Incorrect sign extension for negative numbers.
- Wrong implementation of addition or subtraction operations.

These errors produce incorrect results, especially for negative operands.

4. Correct Debugged Solution

To correct Booth's Algorithm:

- Verify the bit pair (Q_0, Q_{-1}) before every operation.
- Perform **addition** when the bit pair is **01**.
- Perform **subtraction** when the bit pair is **10**.
- Use **arithmetic right shift** instead of logical right shift.
- Preserve the sign bit during every shift.
- Apply proper sign extension for negative numbers.
- Repeat the process exactly **n** times.

These corrections ensure accurate signed multiplication.

5. Optimization Technique

To improve performance:

- Use hardware support for Booth encoding.
- Minimize unnecessary addition and subtraction operations.
- Implement efficient arithmetic shift registers.
- Use pipelining in the multiplication unit for faster execution.
- Verify sign extension automatically using dedicated hardware.

These optimizations reduce execution time and improve multiplication speed.

6. Testing and Validation

Test Case Expected Output

$$5 \times 4 \quad 20$$

$$-5 \times 3 \quad -15$$

$$-6 \times -4 \quad 24$$

$$7 \times -2 \quad -14$$

All positive and negative combinations should produce the correct signed multiplication result.

7. Conclusion

The incorrect output is caused by errors in **bit-pair checking, arithmetic right shifting, and sign extension**. By correctly implementing Booth's Algorithm, preserving the sign bit, and performing proper arithmetic shifts, the multiplication unit produces accurate results for both positive and negative operands while improving overall efficiency.

Question 4

An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

1. Identification of Error / Bug

The main error is **algorithmic inefficiency**, not an incorrect output.

During every iteration:

- The divisor is subtracted from the remainder.
- If the remainder becomes negative, the algorithm immediately restores the previous value by adding the divisor again.
- These repeated restoration operations increase the number of arithmetic operations and clock cycles.

Root Cause: Unnecessary restoration after every unsuccessful subtraction.

2. Debugging Approach and Analysis

Consider the example:

Dividend = 13

Divisor = 3

The restoring division algorithm performs the following operations:

1. Shift the remainder and dividend.
2. Subtract the divisor.
3. If the remainder is positive, store quotient bit = 1.

4. If the remainder is negative, restore the previous remainder by adding the divisor and store quotient bit = 0.
5. Repeat the process until all bits are processed.

During debugging, it is observed that whenever subtraction produces a negative remainder, an additional restoration step is executed. Although the final quotient is correct, the repeated restoration increases execution time and processor workload.

3. Correctness of Debugged Solution

The problem can be solved by replacing the **Restoring Division Algorithm** with the **Non-Restoring Division Algorithm**.

In the Non-Restoring method:

- If the previous remainder is positive, subtract the divisor.
- If the previous remainder is negative, add the divisor in the next cycle instead of restoring immediately.
- Continue the division process without unnecessary restoration.
- Perform only one final correction if the remainder is negative after the last iteration.

This method produces the same quotient and remainder while requiring fewer arithmetic operations.

4. Optimization Techniques Applied

The Non-Restoring Division Algorithm improves system performance because it:

- Eliminates repeated restoration operations.
- Reduces the total number of additions.
- Requires fewer clock cycles.
- Decreases execution time.
- Improves processor throughput.
- Is more suitable for embedded and real-time applications where speed is important.

5. Testing and Documentation

The optimized algorithm was tested using different division operations.

Dividend	Divisor	Expected Quotient	Expected Remainder	Result
13	3	4	1	Correct
20	5	4	0	Correct
25	6	4	1	Correct
18	4	4	2	Correct

- The test results show that both algorithms produce the same output, but the Non-Restoring Division Algorithm completes the operation with fewer arithmetic steps and lower execution time.

Conclusion

The debugging process identified that the performance bottleneck was caused by repeated restoration operations in the Restoring Division Algorithm. Replacing it with the **Non-Restoring Division Algorithm** eliminates unnecessary restoration steps, reduces execution time and hardware operations, and improves the overall efficiency of the embedded system while maintaining accurate quotient and remainder values. This makes the optimized algorithm more suitable for high-speed and real-time processor applications.

A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

1. Identification of Error / Bug

The application gives incorrect results because of the **limited precision (32-bit)** of IEEE 754 single-precision floating-point representation.

Example:

Large Number = **1,000,000.0**

Small Number = **0.000001**

Expected Result:

1,000,000.000001

Actual Result (Single Precision):

1,000,000.0

The small value is lost during addition because of precision limitations.

Root Cause:

- Large difference between exponents.
- Loss of significant digits during exponent alignment.
- Rounding errors after normalization.

2. Debugging Approach and Analysis

The floating-point addition process follows these steps:

Step 1: Compare the exponents of both numbers.

Step 2: Shift the mantissa of the smaller number until both exponents become equal.

Step 3: Add the mantissas.

Step 4: Normalize the result.

Step 5: Round the final result according to IEEE 754 rules.

During debugging, it is observed that when the exponent difference is very large, the mantissa of the smaller number is shifted so many times that its significant bits are discarded. As a result, the smaller value has no effect on the final answer, producing an inaccurate result.

3. Correctness of Debugged Solution

The issue can be corrected by improving floating-point precision.

The following solutions are applied:

- Replace **IEEE 754 single precision (32-bit)** with **double precision (64-bit)**.
- Maintain more significant digits during computation.
- Use proper normalization and IEEE 754 rounding methods.
- Rearrange arithmetic operations by adding numbers of similar magnitude first before adding very large values.

These corrections significantly reduce precision loss and improve computational accuracy.

4. Optimization Techniques Applied

To improve the performance and accuracy of the scientific application:

- Use **64-bit double precision** instead of 32-bit single precision.
- Reduce rounding errors using IEEE 754 rounding modes.
- Add smaller numbers together before adding them to larger numbers.
- Avoid unnecessary floating-point conversions.
- Use high-precision arithmetic libraries for scientific calculations when required.

5. Testing and Documentation

The optimized solution was tested using different floating-point values.

Test Case	Expected Result	Output
$100.5 + 20.25$	120.75	Correct
$1,000,000 + 0.000001$	1,000,000.000001	Correct using double precision
$0.125 + 0.375$	0.5	Correct

- The testing confirms that double precision preserves more significant digits and provides better accuracy than single precision.

Conclusion

The debugging process identified that the inaccurate results were caused by exponent alignment, normalization, and rounding errors in IEEE 754 single-precision representation. Replacing single precision with double precision, improving normalization, and applying suitable arithmetic techniques significantly reduce precision loss and produce more accurate floating-point calculations. Therefore, the optimized solution is more suitable for scientific and engineering applications requiring high numerical accuracy.